

KRCIGE：一个有界工程框架

从认识、资源、控制与工程不可逆性推导软件工程原则

初稿 v0.1

2026 年 8 月 9 日

摘要

软件工程长期积累了大量经验原则，例如 DRY、低耦合、小步迭代、版本控制、自动化测试、显式契约、最少共享状态与持续反馈。这些原则覆盖代码、架构、团队、交付与安全，其表达形式高度多样。本文提出一个探索性的有界工程框架 KRCIGE，尝试用少量基础约束解释并生成这类经验原则。

框架包含四个描述性要素：认识有限（K）、资源有限（R）、控制有限（C）与工程不可逆性（I）；两个规范性要素：工程目标（G）与伦理边界（E）；以及可学习结构条件 Σ 。其中 I 表示行动完成后，以规定质量和可靠性恢复到任务等价状态所需的最低预期成本。本文定义十三条中间原则，给出可复算的发布案例，以 *The Pragmatic Programmer* 100 条 Tips 检查压缩能力，并用 Twelve-Factor App 进行一次内部前瞻性留出核验。

本文的核心主张是：软件工程经验可以表达为在 K、R、C、I 约束下，为实现 G 且满足 E 所形成的条件性策略。框架强调反馈、选择权、复杂度预算、局部化、信息保留与恢复能力，并公开其适用条件、反转条件和验证边界。

目录

1 动机	3
2 基本模型	4
3 推导语义与主张等级	4
4 KRCIGE 的六个要素与适用条件	5
4.1 K：认识有限	5
4.2 R：资源有限	5
4.3 C：控制有限	6
4.4 I：工程不可逆性	6
4.5 G：工程目标	7
4.6 E：伦理边界	8
4.7 Σ ：可学习结构条件	8

目录	2
5 十三条中间原则	8
5.1 P1: 反馈与信息价值	8
5.2 P2: 选择权与最少承诺	9
5.3 P3: 复杂度预算	9
5.4 P4: 局部化与模块化	9
5.5 P5: 显式假设	9
5.6 P6: 信息保留与证据价值	10
5.7 P7: 权威来源与协调语义	10
5.8 P8: 自动化阈值	10
5.9 P9: 状态空间控制	11
5.10 P10: 不可逆性与恢复能力	11
5.11 P11: 安全暴露面与爆炸半径	11
5.12 P12: 知识外部化	11
5.13 P13: 模型持续校正	11
6 几个代表性推导	12
6.1 DRY	12
6.2 YAGNI 与避免预言未来	12
6.3 版本控制	13
6.4 自动化测试	13
6.5 低耦合	13
6.6 显式时间语义	13
7 完整可计算案例：直接发布与分阶段发布	14
7.1 场景、证据与候选策略	14
7.2 期望效用计算	15
7.3 敏感性阈值与可反驳结论	15
8 原则冲突与条件化推导	16
8.1 DRY 与抽象时机	16
8.2 Fail Fast 与 Graceful Degradation	16
8.3 小步迭代与原子不变量	16
8.4 配置化与状态空间	16
8.5 Postel 鲁棒性原则	17

1 动机	3
9 三条进一步原则	17
9.1 证据半衰期原则	17
9.2 保证-假设对偶	17
9.3 副本权威原则	18
10 独立性与边界	18
11 理论的可检验性	18
11.1 内部留出核验	18
11.2 可反驳条件	20
11.3 可量化研究方向	20
12 适用范围	21
13 使用清单	21
14 结论	21
A 对《The Pragmatic Programmer》100 条 Tips 的映射	22
B 符号表	26

1 动机

软件工程拥有数量庞大的经验原则。它们来自不同年代、不同语言、不同组织规模 and 不同技术栈，却经常指向相似的工程行为：减少无关耦合、缩小一次变更的影响范围、保留历史、自动化重复工作、尽快取得反馈、显式表达假设、控制共享状态、让架构保持可修改性。

一个自然的问题是：这些经验是否共享更少、更基础的原因？

本文提出 KRCIGE 框架，试图把软件工程原则组织为以下推导结构：

$$\boxed{\text{现实约束} + \text{目标与价值边界} \rightsquigarrow \text{中间工程原则} \rightsquigarrow \text{具体实践}} \quad (1)$$

KRCIGE 的目标包含三个层次：

- (1) **压缩性**：用较少基础命题解释较多经验原则；
- (2) **生成性**：从基础命题推出新的工程判断；
- (3) **条件性**：解释某条经验原则在什么条件下增强、减弱或需要与另一条原则共同优化。

该框架与若干已有理论传统具有明显联系。Simon 的有限理性强调知识、记忆与计算能力的约束 [1]；Ashby 的必要多样性定律讨论控制器能力与环境扰动之间的关系 [2]；Landauer 研究逻

辑不可逆计算与信息擦除的物理意义 [3]; Parnas 将模块化与可理解性、可修改性联系起来 [4]; SICP 将数据抽象解释为最少承诺原则的一个应用 [5]; FLP 结果展示了模型假设对分布式系统可实现保证的约束 [6]; No Free Lunch 结果提醒我们, 算法优势依赖问题类别所提供的结构 [7]。

本文将这些思想组织为一个面向软件工程实践的统一框架。

2 基本模型

设工程主体面对一个世界状态

$$w \in \mathcal{W}.$$

主体通过观测函数获得信息

$$o = O(w), \quad o \in \mathcal{O}.$$

主体选择行动

$$a \in \mathcal{A}.$$

世界受到行动和外生扰动 $d \in \mathcal{D}$ 的共同作用:

$$w' = T(w, a, d).$$

工程行动消耗资源。记成本向量为

$$c(a) = (c_{\text{time}}, c_{\text{cpu}}, c_{\text{memory}}, c_{\text{money}}, c_{\text{attention}}, \dots).$$

系统拥有有限预算

$$B = (B_{\text{time}}, B_{\text{cpu}}, B_{\text{memory}}, B_{\text{money}}, B_{\text{attention}}, \dots).$$

工程主体依据策略 π 从观测历史选择行动:

$$a_t = \pi(o_{\leq t}).$$

目标函数记为

$$U(w, a),$$

伦理允许的行动集合记为

$$\mathcal{A}_E \subseteq \mathcal{A}.$$

因此, 工程问题可以抽象为

$$\max_{\pi} \mathbb{E}[U] \quad \text{s.t.} \quad c(\pi) \preceq B, \quad a_t \in \mathcal{A}_E. \quad (2)$$

这个形式仅提供骨架。真实软件工程包含多目标、长期反馈、组织互动、非平稳环境和难以量化的价值。KRCIGE 使用该模型表达约束之间的逻辑关系, 而非提供一个直接数值求解器。

3 推导语义与主张等级

为避免把经验性工程判断写成逻辑定理, 本文区分三类主张。

形式蕴含。 符号

$$A \implies B$$

仅表示：在已经明确给出的定义、公理和数学前提下， B 是 A 的严格逻辑后果。删除任一必要前提都可能使蕴含失效。

条件性工程压力。 符号

$$A \rightsquigarrow B$$

表示：在给定工程目标 G 、伦理边界 E 、成本模型和适用条件时， A 提高了策略 B 的相对预期价值。该符号表达可被额外成本、边界条件或竞争目标逆转的决策倾向。本文从 KRCIGE 到 P1–P13、从中间原则到具体实践的路径均属于这一类型。

经验桥接。 含有辅助前提 Ψ 的路径，例如

$$A + \Psi \rightsquigarrow B,$$

表示从约束到实践还依赖心理学、组织行为、工具效果或其他外部实证规律。 Ψ 必须由相应领域证据支持，无法仅凭 KRCIGE 的形式骨架推出。

因此，本文所称“推导”默认指带有公开前提的条件性解释路径。只有显式使用 $\implies$ 时，文本才主张严格形式蕴含；使用 $\rightsquigarrow$ 时，文本主张可检验、可被条件逆转的工程倾向。

4 KRCIGE 的六个要素与适用条件

4.1 K：认识有限

定义 4.1 (认识有限). 若存在两个与工程目标相关的世界状态 $w_1 \neq w_2$ ，使主体当前观测无法区分二者：

$$O(w_1) = O(w_2),$$

则称工程主体具有认识有限性，记为 K 。

概率语言下，可写成

$$H(W | O) > 0.$$

认识有限覆盖需求不完整、未知故障、未来流量、用户真实偏好、第三方系统状态、未来技术变化等现象。Simon 的有限理性工作为这一方向提供了经典理论背景 [1]。

4.2 R：资源有限

定义 4.2 (资源有限). 若至少一种与工程行动相关的资源具有有限预算

$$B_j < \infty,$$

则称系统满足资源有限性，记为 R 。

资源包含机器资源与人类资源。人的注意力、工作记忆、协作带宽和可支配时间同样构成工程预算。资源有限性使工程问题具有选择、排序、估算和权衡的必要性。

4.3 C: 控制有限

定义 4.3 (控制有限). 若存在主体无法直接选择的扰动 d , 并且

$$T(w, a, d_1) \neq T(w, a, d_2)$$

可在相同行动 a 下发生, 则称系统满足控制有限性, 记为 C。

网络、硬件、用户、调度器、第三方依赖、攻击者和组织行为都可以形成外生变量。Ashby 的必要多样性定律描述了控制能力与环境可能性之间的关系 [2]。分布式计算中的 FLP 结果进一步展示了系统模型与可实现保证之间的严格联系 [6]。

4.4 I: 工程不可逆性

工程行动通常具有不对称的执行路径与恢复路径。删除数据、发布版本、执行支付、压缩表示或承诺公共接口都可能容易执行, 同时难以恢复原有信息、系统状态和未来选择。

设当前观测 o 给出行动前状态的信念分布

$$X \sim P(\cdot | o).$$

行动 a 和外生扰动 D 产生行动后状态

$$Y = T(X, a, D).$$

行动发生后, 主体只能利用行动后状态 Y 与保留下来的证据 $z \in \mathcal{Z}$ 进行恢复。恢复策略写成

$$\rho: \mathcal{Y} \times \mathcal{Z} \rightarrow \mathcal{A}^*, \quad \rho \in \Pi_E,$$

其中 Π_E 表示满足伦理边界的恢复策略集合, $\mathcal{A}^*$ 表示有限行动序列。执行 $\rho(Y, z)$ 后得到恢复状态 X^ρ 。恢复策略无法直接读取已经丢失且未被 Y, z 保留的行动前状态。

令

$$D_Q(x, X^\rho) \geq 0$$

表示两个状态在当前工程问题 Q 所关心的信息区分、系统不变量和未来可选行动方面仍然存在的差异。给定允许差异 $\varepsilon \geq 0$ 与最大失败概率 $\delta \in [0, 1]$, 定义如下。

定义 4.4 (工程不可逆性). 行动 a 的工程不可逆性, 是恢复策略仅能使用 Y, z 时, 以至少 $1 - \delta$ 的概率恢复到 ε -任务等价状态所需的最低预期恢复成本:

$$I_{\varepsilon, \delta}(a | o, z) = \inf_{\rho \in \Pi_E} \{ \mathbb{E}[C_{\text{rec}}(\rho; Y, z) | o] : \Pr[D_Q(X, X^\rho) \leq \varepsilon | o, z] \geq 1 - \delta \}.$$

若不存在满足条件的恢复策略, 则规定

$$I_{\varepsilon, \delta}(a | o, z) = +\infty.$$

这里的 $C_{\text{rec}}(\rho) \geq 0$ 是对时间、算力、金钱、注意力和协调成本的预先约定标量化。若这些资源之间不适合直接交换，可以保留成本向量，并以帕累托前沿表示不可逆性；本文采用标量形式以保持推导简洁。

定义中的 $P(X | o)$ 由 K 所描述的当前认识状态给出，状态转移与恢复策略则决定 I 。二者由同一个决策模型连接，同时保持不同的逻辑角色。

$I_{\epsilon, \delta}$ 越大，表示行动越难以在要求的质量和可靠性下恢复。工程目标 G 决定哪些状态差异重要以及可接受的 ϵ, δ ；伦理边界 E 约束可采用的恢复手段。资源有限 R 决定恢复成本能否被预算承受，控制有限 C 影响哪些恢复策略实际可行。

信息损失是工程不可逆性的一个结构性来源。考虑表示变换

$$f: \mathcal{X} \rightarrow \mathcal{Y}.$$

若存在后验概率为正、任务相关但彼此不同的状态

$$x_1 \neq x_2, \quad f(x_1) = f(x_2),$$

并且保留证据 z 也无法区分二者，则只读取 $f(X), z$ 的恢复策略无法保证精确恢复，相应的 $I_{0,0}$ 为无穷大。保存原始值、版本历史、日志或备份可以扩充 z ，从而降低恢复负担。

例如，

$$(09:00, \text{Asia/Shanghai}) \mapsto 09:00$$

会消去时区语义；

$$3.1415926 \mapsto 3.14$$

会消去精度；

$$\text{DatabaseTimeout} \mapsto \text{false}$$

会消去错误类型和诊断上下文。类似地，无备份的数据删除、已经产生外部影响的支付和被广泛依赖的公共接口承诺，也会产生较高的恢复成本或无法消除的残余差异。

K 衡量行动前仍然存在的关键未知， I 衡量行动后恢复任务相关状态的困难；二者可以独立变化，第 10 节给出区分它们的反例。Landauer 对逻辑不可逆计算与信息擦除的研究为其中的信息损失情形提供了物理层面的背景 [3]。

4.5 G：工程目标

定义 4.5 (工程目标). 工程目标 G 给出对系统状态和行动的偏好。本文采用宽泛表述：

$$G: \quad \text{在约束下持续、可靠地交付预期价值。}$$

G 提供规范方向。 K 、 R 、 C 、 I 描述工程世界的约束， G 决定工程主体希望在这些约束中优化什么。

4.6 E: 伦理边界

定义 4.6 (伦理边界). 伦理边界 E 定义允许行动集合

$$\mathcal{A}_E \subseteq \mathcal{A}.$$

工程策略的行动满足

$$a_t \in \mathcal{A}_E.$$

这一层用于表达安全、隐私、公平、责任和社会伤害等约束。工程目标可以具有多个量化指标，伦理边界为部分行动提供硬约束或高优先级约束。

4.7 Σ : 可学习结构条件

认识有限本身只能说明存在未知量。反馈的价值还依赖问题域存在稳定结构。

定义 4.7 (可学习结构条件). 若当前观测、历史数据、实验结果或局部模型对未来相关结果具有可利用的预测信息，则记问题域满足

$$\Sigma.$$

可以用一个很弱的形式表达：

$$\mathcal{I}_{\text{Shannon}}(Z; Y) > 0,$$

其中 Z 是可获得的证据， Y 是未来关注结果。

No Free Lunch 类结果说明，算法优势依赖问题类别中的结构性假设 [7]。因此 Σ 充当 KR-CIGE 的适用条件：工程学习依靠世界中可重复利用的规律。逻辑上，K、R、C 描述能力边界，I 描述状态转移后的恢复边界，G、E 提供规范方向， Σ 限定学习何时能够积累价值。

5 十三条中间原则

为减少从 KRCIGE 到具体实践的重复推导，本文定义十三条中间原则。

5.1 P1: 反馈与信息价值

派生原则 5.1 (反馈价值原则). 在 $K + R + \Sigma + G$ 条件下，当一项观测的预期决策价值高于获取成本时，应优先获得该观测。

经典的 value of information 形式为

$$\text{VOI}(O) = \mathbb{E}_O \left[\max_a \mathbb{E}(U \mid O, a) \right] - \max_a \mathbb{E}(U \mid a).$$

当

$$\text{VOI}(O) > \text{Cost}(O)$$

时，观测具有正工程价值。

由此可导出原型、示踪子弹、自动测试、性能测量、用户反馈、端到端验证和迭代需求发现。

5.2 P2: 选择权与最少承诺

派生原则 5.2 (选择权原则). 在 $K + I + R + G$ 条件下, 未来不确定性越高、候选行动的工程不可逆性越强, 保留选择权的价值越高。

可以粗略定义等待的期权价值:

$$V_{\text{wait}} = V_{\text{future choice}} - C_{\text{delay}}.$$

当等待带来的未来选择价值覆盖延迟成本时, 推迟不可逆承诺具有正价值。

SICP 将数据抽象解释为最少承诺原则的应用: 抽象屏障允许具体表示选择延后, 从而保留系统灵活性 [5]。

5.3 P3: 复杂度预算

派生原则 5.3 (复杂度预算原则). 复杂度消耗有限资源。新增复杂度应带来足以覆盖实现、理解、运行和变更成本的工程价值。

可以使用总成本表达:

$$C_{\text{total}} = C_{\text{implementation}} + C_{\text{understanding}} + C_{\text{operation}} + C_{\text{change}}.$$

该原则支持简单设计、渐近复杂度分析、减少无收益抽象、控制继承层次和避免无价值技术迁移。

5.4 P4: 局部化与模块化

派生原则 5.4 (局部化原则). 在 $K + R + C + G$ 条件下, 应缩小一次变更要求同时理解、协调和控制的范围。

可以把模块化目标写成

$$\min (\mathbb{E}[C_{\text{propagation}}] + C_{\text{boundary}}).$$

第一项是变化传播成本, 第二项是接口、部署、通信和边界管理成本。

Parnas 的经典工作将模块划分与灵活性、可理解性联系起来 [4]。该表达同时允许解释单体模块化、服务边界和微服务粒度的权衡。

5.5 P5: 显式假设

派生原则 5.5 (显式假设原则). 在 $K + C + I$ 条件下, 跨越模块、团队或时间边界的重要假设, 应尽量编码为显式、可验证的数据、类型、契约或测试。

该原则产生 Design by Contract、断言、schema、领域类型、显式状态、属性测试与可执行规范。

5.6 P6: 信息保留与证据价值

派生原则 5.6 (信息保留原则). 在满足伦理边界的前提下, 若某项信息未来被需要的概率为 p , 丢失后增加的恢复或决策损失为 L , 保存成本为 S , 保存带来的安全与隐私风险为 Q , 则在

$$pL > S + Q$$

时保存具有正期望价值。

该原则支持版本历史、结构化日志、审计轨迹、原始事件、诊断上下文和高精度中间表示。被伦理边界禁止的保存行为首先从可行集合中排除, 其余方案再比较未来决策价值、恢复负担、存储成本与风险。

5.7 P7: 权威来源与协调语义

派生原则 5.7 (权威来源原则). 当同一事实存在多个副本时, 应显式给出权威来源或冲突协调语义。

其推导来自

$$R + C + I + G.$$

多副本同步消耗资源; 外部更新和并发变化增加分叉概率; 分叉会削弱“哪个副本表达当前事实”的信息。

这一原则给出 DRY 的一个更一般版本:

同一权威知识需要明确 authority 或 reconciliation。

5.8 P8: 自动化阈值

派生原则 5.8 (自动化原则). 设人工流程执行 n 次的预期成本为

$$nC_h + R_h,$$

自动化方案成本为

$$F + nC_a + R_a.$$

当

$$F + nC_a + R_a < nC_h + R_h$$

时, 自动化具有正期望收益。

这里 F 为自动化固定成本, C_h, C_a 为每次执行成本, R_h, R_a 为失败与变异风险成本。

5.9 P9: 状态空间控制

派生原则 5.9 (状态空间原则). 在 $K + R + C$ 条件下, 工程设计应控制需要推理、测试和协调的可达状态空间。

共享可变状态会增加可能 interleaving 的数量。消息传递、不可变数据、事务边界、actor 和状态机都可以作为状态空间约束技术。

该原则允许受控共享状态。数据库事务、锁、共识协议和明确的一致性模型都可以提供协调语义。

5.10 P10: 不可逆性与恢复能力

派生原则 5.10 (可恢复性原则). 行动的工程不可逆性越高, 预防错误和降低恢复负担的机制通常具有越高价值。

设防护机制的成本为 C_{guard} , 行动发生错误的概率为 p_{error} , 防护机制能够减少的错误后损失为 $\Delta L(I_{\varepsilon, \delta})$ 。当

$$C_{\text{guard}} < p_{\text{error}} \Delta L(I_{\varepsilon, \delta})$$

时, 防护机制具有正期望价值。在其他条件相近时, $I_{\varepsilon, \delta}$ 上升会提高 review、dry-run、备份、分阶段发布、幂等、授权和审计等机制的价值。

数据库迁移、支付、权限变更和数据删除都可以依据该原则配置不同强度的验证与恢复机制。

5.11 P11: 安全暴露面与爆炸半径

派生原则 5.11 (安全约束原则). 在 $R + C + I + E$ 条件下, 应降低攻击者可操纵的状态空间, 并限制单次失效能够造成的最大伤害。

这产生最小权限、较小攻击面、快速安全修补、隔离和数据最小化等实践。

5.12 P12: 知识外部化

派生原则 5.12 (知识外部化原则). 重要工程知识应具有超越单个人脑的持久表达。

推导路径为

$$\begin{aligned} K + R + I + G &\rightsquigarrow \text{个人知识容量有限且会随时间流失,} \\ &\rightsquigarrow \text{文档、代码、测试、术语表、评审与版本历史.} \end{aligned}$$

5.13 P13: 模型持续校正

派生原则 5.13 (模型更新原则). 计划、需求、架构和估算均可视为关于世界的模型。获得新证据后, 应根据证据质量与更新成本修正模型。

写成

$$M_t \xrightarrow{O_{t+1}} M_{t+1}.$$

该原则由

$$K + C + \Sigma + G$$

驱动，并支持滚动计划、持续需求发现、性能校准和基于结果的技术选择。

6 几个代表性推导

6.1 DRY

设同一业务知识 q 被复制到 n 个位置：

$$q_1, q_2, \dots, q_n.$$

每次规则变化需要同步更新多个副本，产生维护成本：

$$C_{\text{update}} \propto n.$$

由于 C，无法保证所有修改在所有副本上同步发生；由于 I，副本分叉以后需要付出重新确认权威、协调冲突和恢复一致性的成本；由于 R，持续人工同步受到预算约束。

因此：

$$R + C + I + G \rightsquigarrow P7 \rightsquigarrow \text{DRY}.$$

这一推导把 DRY 的核心对象定位为“权威知识”。两个代码片段拥有相似文本，同时表达独立业务语义时，可以保持独立演化。

6.2 YAGNI 与避免预言未来

未来需求属于当前未知：

$$K.$$

提前实现未来功能立即消耗资源：

$$R.$$

加入接口、数据结构和依赖会收缩未来可选行动，并提高恢复到原有设计空间的成本：

$$I.$$

因此，在需求尚未获得足够证据时：

$$K + R + I + G \rightsquigarrow P2 + P3 \rightsquigarrow \text{延迟未经证实的结构承诺}.$$

6.3 版本控制

代码修改受到 C：误操作、错误假设、依赖变化和协作冲突均可能发生。若直接覆盖旧状态且缺少恢复证据 z ，行动的 $I_{\varepsilon,\delta}$ 会升高；人工保留历史又持续消耗 R。

因此：

$$C + I + R + G \rightsquigarrow P6 + P10 + P8 \rightsquigarrow \text{版本控制}.$$

版本控制把历史纳入可用证据 z ，同时提供恢复路径并降低历史保存的边际成本，因此能够系统性降低代码变更的工程不可逆性。

6.4 自动化测试

当前代码正确性受到 K；未来输入和依赖行为受到 C；人工回归受到 R。

在 Σ 成立时，测试结果对未来故障具有信息价值：

$$K + C + R + \Sigma + G \rightsquigarrow P1 + P5 + P8 \rightsquigarrow \text{自动化测试}.$$

测试还能把已发现行为规则外部化：

$$P12.$$

因此“每个 Bug 只需被发现一次”可进一步表示为

$$\text{Bug 证据} \rightsquigarrow \text{回归测试} \rightsquigarrow \text{长期保存新获得的不变量知识}.$$

6.5 低耦合

若一个模块变化会迫使大量模块共同修改，则一次变化的知识需求、协调成本和不可控传播范围同时增加。

因此：

$$K + R + C + G \rightsquigarrow P4 \rightsquigarrow \text{低耦合、封装与局部化}.$$

进一步加入接口边界成本，可获得条件化结论：

$$\min (C_{\text{change propagation}} + C_{\text{boundary}}).$$

该表达可以同时评价模块化单体与分布式服务架构。

6.6 显式时间语义

设系统需要表达一个确定时刻和其业务时区语义：

$$x = (t, z).$$

若传输为

$$f(t, z) = t_{\text{local}},$$

多个 (t, z) 可能产生相同字符串。I 表明在缺少额外证据时无法精确恢复原有时间语义；K 表明接收方当前无法确定发送方采用了哪一种隐式上下文。

因此：

$K + I \rightsquigarrow P5 + P6 \rightsquigarrow$ 显式携带 Instant、offset、zone 或明确的时间语义。

7 完整可计算案例：直接发布与分阶段发布

本节给出一个端到端数值案例，使 K、R、C、I、G、E 与 Σ 共同进入同一个决策。数值均为演示性输入，作用是展示计算方法和可反驳阈值。

7.1 场景、证据与候选策略

团队准备发布一项会改变第三方客户端交互方式的功能。试点前采用缺陷概率先验

$$q \sim \text{Beta}(1, 9).$$

在 40 个集成场景中观察到 3 个缺陷和 37 个成功结果，得到后验

$$q \mid o \sim \text{Beta}(4, 46), \quad \mathbb{E}[q \mid o] = \frac{4}{50} = 0.08.$$

这一分布表达 K：团队仍无法确定正式流量是否会触发缺陷。第三方客户端行为和真实流量属于 C，团队只能通过发布策略限制其影响。

比较两个候选策略：

- a_D ：直接向全部用户发布；
- a_S ：使用 feature flag，先向 10% 用户发布，观察后再扩大范围。

所有价值和成本统一换算为“工程小时等价成本”（EH）。恢复演练在

$$\varepsilon = 0, \quad \delta = 0.01$$

的要求下得到表 1 中的输入。 $I_D = 80$ 表示直接发布后，以至少 99% 的成功概率恢复到任务等价状态所需的最低成本为 80 EH；分阶段发布保留旧路径、发布标记和局部事件证据，因此 $I_S = 6$ 。

R 给出两个预算约束：前置实施预算为 50 EH，事故恢复预算为 100 EH。因此两个策略当前都可执行。E 要求观测与恢复过程不得新增采集个人敏感数据；表中的两个恢复策略已经在该约束下计算。 Σ 表示试点错误信号对正式流量结果具有稳定预测价值；它支持把分阶段发布的缺陷影响限制在 10% 范围。若该稳定关系不成立，表中的 $L_S = 60$ 便失去依据，需要重新估计。

表 1: 数值案例的演示性输入

变量	直接发布 a_D	分阶段发布 a_S
成功交付价值 V	240	240
实现与发布成本 C	20	35
延迟价值损失 C_{delay}	0	12
缺陷发生后的直接损失 L	600	60
工程不可逆性 $I_{0,0.01}$	80	6

7.2 期望效用计算

使用后验均值 $\bar{q} = 0.08$ ，直接发布的期望效用为

$$\mathbb{E}[U_D] = (1 - \bar{q})V - C_D - \bar{q}(L_D + I_D) \quad (3)$$

$$= 0.92 \times 240 - 20 - 0.08 \times (600 + 80) \quad (4)$$

$$= 146.40. \quad (5)$$

分阶段发布的期望效用为

$$\mathbb{E}[U_S] = (1 - \bar{q})V - C_S - C_{\text{delay}} - \bar{q}(L_S + I_S) \quad (6)$$

$$= 0.92 \times 240 - 35 - 12 - 0.08 \times (60 + 6) \quad (7)$$

$$= 168.52. \quad (8)$$

因此

$$\mathbb{E}[U_S] - \mathbb{E}[U_D] = 22.12 > 0,$$

在给定输入和预算下选择 a_S 。

7.3 敏感性阈值与可反驳结论

两个策略的期望效用差可以整理为

$$\mathbb{E}[U_S - U_D] = -(C_S + C_{\text{delay}} - C_D) \quad (9)$$

$$+ q[(L_D + I_D) - (L_S + I_S)] \quad (10)$$

$$= -27 + 614q. \quad (11)$$

因此分阶段发布占优的阈值为

$$q > \frac{27}{614} \approx 0.0440.$$

该结论具有明确的条件反转方向：当缺陷概率低于约 4.40% 时，直接发布在同一价值模型下占优；当前置预算低于 35 EH 时， a_S 因 R 约束退出可行集合；当 Σ 不成立时，需要撤销对

$L_S = 60$ 的使用；当 E 排除试点所需观测方式时，需要重新设计恢复证据 z 。这个案例中的路径属于

$K + C + I + R + \Sigma + G + E \rightsquigarrow P1 + P2 + P6 + P10 + P13 \rightsquigarrow$ 在阈值条件内选择分阶段发布。

8 原则冲突与条件化推导

KRCIGE的一个重要用途是处理经验原则之间的张力。此时目标是寻找多个约束共同作用下的最优区域。

8.1 DRY 与抽象时机

DRY 主要由 $P7$ 支持；抽象时机主要由 $P2$ 支持。

当多个实现仍缺少足够证据证明它们承载同一稳定知识时， K 较强，选择权价值较高。随着重复实例增加、共同变化模式变得清晰， $P7$ 的收益逐渐上升。

由此得到：

先观察共同变化，再抽取共同权威知识。

8.2 Fail Fast 与 Graceful Degradation

靠近错误源时，继续传播错误状态会损失诊断信息，因此 $P5 + P6$ 支持快速暴露。

靠近用户价值边界时，服务整体失效可能扩大损失，因此 $P10 + P11$ 支持隔离、降级和恢复。

可形成分层规则：

局部尽早暴露错误，系统边界限制故障传播。

8.3 小步迭代与原子不变量

$P2$ 支持较小承诺； $P10$ 支持可恢复步骤。若一个不变量要求若干变化共同生效，则最小安全步骤由不变量边界决定。

因此得到：

选择能够独立保持关键不变量的最小可恢复步骤。

8.4 配置化与状态空间

外部配置通过 $P2$ 保留策略选择权，同时每增加一个运行时参数都会扩大 $P9$ 所描述的状态空间。

因此：

高变化频率策略适合显式配置；高稳定度不变量适合固化在类型、schema 与代码约束中。

8.5 Postel 鲁棒性原则

协议接收方容忍更多输入可以吸收 C 带来的外部实现差异；同时，过度容忍会降低发送方获得错误反馈的机会，并允许歧义长期存在。RFC 9413 对鲁棒性原则的长期互操作性影响进行了系统讨论 [11]。

KRCIGE 可将这一张力表示为

$$C \text{ 与 } K+I$$

之间的权衡。

对于缺少协调能力的旧生态，兼容容忍具有较高价值；对于可主动维护的新协议，严格验证能够形成更高质量的反馈与更小的协议状态空间。

9 三条进一步原则

以下原则超出 *The Pragmatic Programmer* Tip 表中的直接表达，用于展示框架的生成性。

9.1 证据半衰期原则

有些故障证据会迅速消失。例如一次请求的原始 payload、线程调度、第三方响应和内存现场。设时刻 t 可用于恢复的证据为 z_t 。在没有新增证据且证据集合随时间收缩时，通常有

$$z_{t+1} \subseteq z_t \rightsquigarrow I_{\varepsilon, \delta}(a \mid o, z_{t+1}) \geq I_{\varepsilon, \delta}(a \mid o, z_t).$$

定义证据价值

$$V_e(t)$$

随时间下降。若

$$\frac{dV_e}{dt} \ll 0,$$

则应靠近产生点采集证据。

由此得到：

越难重现的证据，越应靠近事件源自动捕获。

9.2 保证-假设对偶

任何强保证都建立在一组系统假设上。将保证记为 G_u ，假设集合记为

$$\mathcal{H} = \{H_1, \dots, H_n\}.$$

则工程评审应同时记录：

$$G_u \leftrightarrow \mathcal{H}.$$

FLP 结果提供了这一思想的经典极端案例：异步性和故障模型直接约束可实现的共识保证 [6]。

9.3 副本权威原则

允许数据拥有多个副本，同时要求：

replication $\rightsquigarrow$ authority 或 reconciliation semantics.

cache、read replica、event projection 和多地域副本均可在这一原则下分析。

10 独立性与边界

下表用反例区分各要素，避免把不同约束合并为笼统的“困难”。

区分	反例
K / R	不可观测隐藏位在资源充足时仍然未知；完全可观测的有限状态机仍消耗时间、内存和能量。
K / C	主体可以知道天气而无法改变天气，也可以不知道寄存器旧值而直接控制其下一状态。
I / K	已知删除生产数据的后果仍可能无法恢复；不了解新功能效果时，沙箱和 feature flag 可以保持低 I。
I / R	I 给出达到恢复要求的最低负担，R 给出主体可支配的预算；相同恢复任务对不同预算主体具有不同可行性。
I / C	C 限定可执行的恢复策略集合 Π_E ；第三方接口、人员行为和外部传播会改变 I。

前文的多对一变换进一步给出信息擦除的结构性案例。

KRCI 描述约束，G 指定优化方向，E 保留不可交换的伦理、安全和责任边界。涉及心理、团队、组织或具体工具效果的路径标记为辅助经验前提 Ψ 。该标记只声明路径依赖外部证据；每项使用都需要具体主张、适用范围和相应实证支持，含 Ψ 的路径不计作 KRCIGE 核心独立解释。

11 理论的可检验性

一个只对既有原则进行事后解释的框架具有有限科学价值。KRCIGE 需要接受独立语料和预测测试。

11.1 内部留出核验

The Pragmatic Programmer 100 Tips 是构造语料，其完整映射位于附录；Twelve-Factor App 的 12 个因素构成第一组内部留出材料 [10]，没有用于新增基础要素或修改 P1–P13。

本轮验证采用以下顺序：

1. 冻结 KRCIGE 定义、P1-P13 与推导符号；
2. 只依据 Twelve-Factor 官方索引中的因素名称和一句式副标题，记录主要约束、预测路径及方向条件；
3. 再读取每个因素的官方详细说明，检查预测机制与方向是否出现；
4. 核验期间不新增中间原则，也不回写预测路径。

分类规则预先固定为：若官方说明清楚支持冻结预测的主要机制和方向，记为“直接支持”；若支持主要机制，但方向条件或主要路径只有部分吻合，记为“部分支持”；若缺少相关机制，记为“未支持”；若出现相反机制或方向，记为“方向相反”。该分类同时记录核验差异以及是否需要新的基础要素。

表 2: Twelve-Factor App 留出集预测与核验结果

#	留出因素	冻结预测	官方说明核验与分类
1	Codebase	$P7 + P6$ ；部署分叉增多时，权威来源与版本历史价值上升。	官方要求一个代码库对应一个应用、由版本控制追踪并支持多个部署，覆盖权威来源和历史保留。分类：直接支持。
2	Dependencies	$P5 + P7 + P10$ ；执行环境差异越大，显式声明和隔离的价值越高。	官方要求完整声明、隔离依赖，并以未来机器可能缺包或版本不兼容解释其价值。分类：直接支持。
3	Config	$P2 + P5 + P9$ ；部署间变化越频繁，配置与代码分离的价值越高。	官方把配置定义为部署间变化项，要求独立、正交管理，并讨论组合爆炸。分类：直接支持。
4	Backing services	$P4 + P5 + P10$ ；外部服务替换和故障概率上升时，附着式边界价值上升。	官方要求以资源句柄连接服务，并给出数据库故障后从备份恢复、重新挂接且无需改代码的例子。分类：直接支持。
5	Build, release, run	$P4 + P6 + P10$ ；发布恢复要求越高，阶段分离和不可变发布账本越有价值。	官方严格区分三个阶段，要求发布具有唯一 ID、保持追加式不可变，并明确支持快速回滚。分类：直接支持。
6	Processes	$P9 + P4 + P10$ ；重启和调度越不可控，无共享、无状态进程的价值越高。	官方要求进程无状态且无共享，并以重启、迁移和不同进程处理后续请求说明本地状态不可靠。分类：直接支持。
7	Port binding	$P4 + P5$ ；运行容器差异增大时，自包含服务和显式端口契约价值上升。	官方支持自包含服务和端口契约，机制吻合；详细说明没有直接比较容器差异变化时的策略强度。分类：部分支持。
8	Concurrency	$P9 + P3 + P13$ ；负载与工作类型增加时，按进程类型分解和横向扩展价值上升。	官方以进程类型表达工作差异，以无共享和水平分区解释可靠扩展，并讨论纵向扩展限制。分类：直接支持。
9	Disposability	$P10 + P4$ ；终止和迁移越频繁，快速启动、优雅退出与幂等价值上升。	官方直接以弹性扩缩、部署、硬件突然失效和任务重入说明快速启动、优雅退出与幂等。分类：直接支持。

#	留出因素	冻结预测	官方说明核验与分类
10	Dev/prod parity	$P1 + P5 + P13$; 环境差异越大, 生产特有错误和反馈延迟越高。	官方列出时间、人员和工具三类差距, 并说明后端服务差异会使测试通过的代码在生产失败。分类: 直接支持。
11	Logs	$P1 + P6 + P12$; 运行时不确定性越高, 事件证据的诊断与历史价值越高。	官方把日志定义为事件流, 并明确用于历史检索、趋势分析和主动告警。分类: 直接支持。
12	Admin processes	$P4 + P8 + P10$; 一次性高风险任务应与常驻流程分离并沿用可恢复发布路径。	官方支持一次性进程及相同代码、配置和依赖, 机制部分吻合; 其主要论证更集中于环境一致性和同步, 未直接给出不可逆性强度关系。分类: 部分支持。

分类结果为 10 项直接支持、2 项部分支持, 未出现“未支持”或“方向相反”, 也没有触发新基础约束。该计数仅描述本次分类分布, 不换算为命中率; Port binding 与 Admin processes 保留为部分支持, 以呈现冻结预测与官方论述的差异。

证据等级限于内部前瞻性核验: 单一评审者可能接触过相关思想, 材料来自同一文档家族, 多标签路径也具有自由度。下一步需要公开冻结预测、盲态独立编码、评审一致性统计和第二类领域材料。

11.2 可反驳条件

以下结果会削弱 KRCIGE:

- 1. 大量核心工程原则需要持续引入彼此无关的新基础约束;
- 2. 同一个基础条件经常同时推出方向相反且无法由成本、目标或层级区分的策略;
- 3. KRCIGE 对原则适用条件的预测与实证结果长期系统性相反;
- 4. 中间原则数量持续接近经验原则数量, 压缩优势消失;
- 5. Σ 、 Ψ 等辅助条件不断扩张, 使任意观察都能被事后吸收。

这些标准能够约束框架复杂度, 降低循环解释的风险。

11.3 可量化研究方向

跨项目研究应使用可观测代理、区间或后验分布表示 K、R、C、I, 并公开测量误差与模型假设。恢复演练可以提供已知恢复策略的成本和成功率; 信息丢失证明可以提供恢复能力的结构

性下界。可检验的方向性预测包括：

$k \uparrow \rightsquigarrow$ 在其他条件相近时，小步实验和可逆性投入上升， (12)

$i \uparrow \rightsquigarrow$ 在防护有效时，review、备份、授权与审计价值上升， (13)

$c \uparrow \rightsquigarrow$ 冗余、隔离、超时和恢复机制投入上升， (14)

$r \uparrow \rightsquigarrow$ 复杂度预算趋紧、自动化阈值发生变化。 (15)

项目历史、事故报告、设计实验和组织对照研究可以用于检验这些关系。

12 适用范围

KRCIGE当前覆盖程序与 API、数据表示、架构边界、测试观测、并发与分布式系统、发布恢复、部分安全工程和需求迭代。团队心理、组织政治、激励机制、职业成长与宏观社会伦理需要多主体或相应领域理论； Ψ 与 E 只提供显式接口。

13 使用清单

应用框架时依次回答：

1. **K**：当前有哪些关键未知量？
2. **R**：最稀缺的资源是什么？
3. **C**：哪些变量来自外部、无法直接控制？
4. **I**：行动后恢复相关信息、状态和未来选择需要多大成本，能够达到什么质量与可靠性？
5. **G**：当前真正优化的价值是什么？
6. **E**：哪些行动受到伦理、安全与责任边界约束？
7. Σ ：哪些反馈具有稳定的预测价值？

随后再选择反馈、模块化、自动化、冗余、测试、日志、配置或版本控制等技术。

14 结论

KRCIGE将软件工程实践组织为“描述性约束+规范方向+适用条件”到条件性策略的推导。其主要贡献是工程不可逆性的恢复成本定义、严格蕴含与工程压力的语义区分，以及由十三条中间原则形成的可复用解释层。

数值案例展示了条件反转如何计算；100 Tips 映射与 Twelve-Factor 内部留出核验提供探索性材料。框架的进一步可信度取决于更严格的 Ψ 管理、独立复制、反例搜索，以及覆盖目标冲突和资源分配的多主体组织扩展。

A 对《The Pragmatic Programmer》100 条 Tips 的映射

The Pragmatic Programmer 20 周年版公开列出 100 条 Tips [8, 9]。本节将其作为经验语料，对 KRCIGE 的压缩能力进行初步检验。这里的“推导”表示解释路径；其中涉及心理学、组织行为和职业文化的条目，会额外标记辅助经验前提 Ψ 。

为便于阅读，使用以下缩写：

P_1 = 反馈价值, P_2 = 选择权, P_3 = 复杂度预算,
 P_4 = 局部化, P_5 = 显式假设, P_6 = 信息保留,
 P_7 = 权威来源, P_8 = 自动化, P_9 = 状态空间,
 P_{10} = 可恢复性, P_{11} = 安全约束, P_{12} = 知识外部化,
 P_{13} = 模型更新.

表 3: 100 条 Tips 与 KRCIGE 的初步推导路径

#	Tip (中文概括)	路径
1	认真打磨自己的专业能力	$K + R + G + \Psi \rightsquigarrow P_{12} \rightsquigarrow$ 持续提高能力与知识质量。
2	思考你的工作	$K + \Sigma + G \rightsquigarrow P_1 + P_{13} \rightsquigarrow$ 持续反思并更新工作模型。
3	你拥有主动权	$C + G \rightsquigarrow$ 识别可控行动子集 $\rightsquigarrow P_{13}$ 。
4	提供可选方案，别拿整脚借口搪塞	$C + K + G \rightsquigarrow P_2 \rightsquigarrow$ 在可控范围提供选择与代价。
5	不要容忍已经存在的缺陷、糟糕设计和错误决定	$C + I + R + G \rightsquigarrow P_{10} + P_3 \rightsquigarrow$ 趁上下文与修复成本仍低时处理退化。
6	主动推动有益的改变发生	$C + G + \Psi \rightsquigarrow$ 从可控局部形成反馈与扩散。
7	持续关注系统的整体目标和影响	$K + R + G \rightsquigarrow P_4 \rightsquigarrow$ 局部决策同时检查系统级目标。
8	把质量作为明确的需求来讨论	$K + R + G \rightsquigarrow P_3 + P_{13} \rightsquigarrow$ 将质量、成本与价值显式建模。
9	持续投资你的知识和技能	$K + C + R + G \rightsquigarrow P_{12} + P_{13}$ 。
10	批判性地分析你读到和听到的内容	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
11	像编写代码一样清晰、简洁、可维护地使用自然语言	$K + I \rightsquigarrow P_5 + P_{12} \rightsquigarrow$ 语言同样承载可丢失的工程语义。
12	你说什么重要，你怎么说同样重要	$K + I \rightsquigarrow P_6 + P_{12}$ 。
13	把文档直接构建到系统和工作流中	$K + I + R + C \rightsquigarrow P_{12} + P_8$ 。
14	好设计比坏设计更容易修改	$K + I + R + C + G \rightsquigarrow P_2 + P_4$ 。
15	DRY——避免重复表达同一份知识	$R + C + I + G \rightsquigarrow P_7$ 。
16	让复用变得容易	$R + G \rightsquigarrow P_3 + P_7 + P_4$ 。
17	消除无关事物之间的影响	$K + R + C + G \rightsquigarrow P_4$ 。
18	尽量保留调整和撤销决策的空间	$K + I + G \rightsquigarrow P_2$ 。
19	根据实际问题和证据选择技术，避免盲目追逐潮流	$K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}$ 。

#	Tip (中文概括)	路径
20	尽早构建可运行的局部方案, 用实际结果验证方向	$K + C + \Sigma + R + G \rightsquigarrow P_1 + P_2$ 。
21	用原型验证理解、需求和技术方案	$K + \Sigma + R + I + G \rightsquigarrow P_1 + P_2$ 。
22	让程序贴近真实业务和问题背景	$K + I + R \rightsquigarrow P_5 + P_{12} + P_3$ 。
23	用估算提前暴露不确定性和风险	$K + R + G \rightsquigarrow P_{13} + P_3$ 。
24	根据代码验证结果持续调整进度计划	$K + C + \Sigma + G \rightsquigarrow P_{13}$ 。
25	用纯文本保存知识	$I + K + G \rightsquigarrow P_6 + P_{12} + P_2$, 具体介质选择依赖 Ψ 。
26	用命令 Shell 组合和自动化工作流程	$R + G + \Psi \rightsquigarrow P_8 + P_3$ 。
27	熟练掌握编辑器, 让编辑操作高效而自然	$R + G + \Psi \rightsquigarrow P_3$ 。
28	始终使用版本控制	$C + I + R + G \rightsquigarrow P_6 + P_{10} + P_8$ 。
29	修复问题, 别追究责任	$R + K + G + \Psi \rightsquigarrow P_1 + P_{12}$ 。
30	保持冷静	$R + G + \Psi \rightsquigarrow$ 保护有限认知资源。
31	先写一个会失败的测试, 再修复代码	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
32	仔细读懂错误消息	$K + I + G \rightsquigarrow P_6 + P_1$ 。
33	先检查查询、数据和假设, 别急着认为 <code>select</code> 出错	$K + \Sigma + \Psi \rightsquigarrow P_1$, 结合成熟组件的经验先验。
34	别想当然, 用证据验证	$K + \Sigma + G \rightsquigarrow P_1$ 。
35	学一门文本处理语言	$R + G + \Psi \rightsquigarrow P_8 + P_3$ 。
36	承认软件不可能完美, 持续改进它	$K + R + C + G \rightsquigarrow P_3$ 。
37	用明确的前置条件、后置条件和不变量设计代码	$K + C + I \rightsquigarrow P_5$ 。
38	尽早发现严重错误并停止执行, 避免继续产生错误结果	$C + I + G \rightsquigarrow P_5 + P_6 + P_{10}$ 。
39	用断言检查不变量, 尽早捕获不应发生的状态	$K + C + \Sigma \rightsquigarrow P_5 + P_1$ 。
40	完成已经开始的工作, 或明确结束它的方式	$R + I + G + \Psi \rightsquigarrow P_3 + P_{12}$ 。
41	把变化和影响限制在局部范围内	$K + R + C + G \rightsquigarrow P_4$ 。
42	每次只做小幅、可验证的改动	$K + C + I + \Sigma + G \rightsquigarrow P_1 + P_2 + P_{10}$ 。
43	不要假设未来需求和技术会怎样, 依据当前证据做可调整的决策	$K + I + R + G \rightsquigarrow P_2 + P_3$ 。
44	减少模块之间的依赖, 让代码更容易修改	$K + R + C + G \rightsquigarrow P_4$ 。
45	让对象直接执行操作, 通过行为接口协作, 减少对内部状态的询问	$K + R + C \rightsquigarrow P_4 + P_5$ 。

#	Tip (中文概括)	路径
46	避免让方法调用形成过长的链条	$K + R + C \rightsquigarrow P_4。$
47	避免全局数据	$K + C + R \rightsquigarrow P_9 + P_4。$
48	通过 API 管理需要全局共享的重要资源	$K + C + I \rightsquigarrow P_5 + P_4 + P_9。$
49	编程要写代码, 但程序最终处理的是数据	$K + I + G \rightsquigarrow P_5 + P_6。$
50	减少状态的集中持有, 让状态沿数据流传递	$K + C + I \rightsquigarrow P_5 + P_9。$
51	警惕继承带来的耦合和维护成本	$R + K + C + G \rightsquigarrow P_3 + P_4。$
52	优先用接口表达不同实现之间的多态关系	$K + I + G \rightsquigarrow P_2 + P_4。$
53	优先通过服务委托和组合复用功能, 减少对继承的依赖	$K + I + R + G \rightsquigarrow P_2 + P_4 + P_3。$
54	用 mixin 共享可复用功能	$R + G + \Psi \rightsquigarrow P_7 + P_4。$
55	让应用通过外部配置适配不同环境	$K + C + I + G \rightsquigarrow P_2 + P_5 + P_9。$
56	分析 workflow, 找出可以并发执行的部分	$R + K + \Sigma + G \rightsquigarrow P_1 + P_9。$
57	尽量避免共享可变状态, 因为它容易导致不一致和并发错误	$K + R + C \rightsquigarrow P_9。$
58	随机失败往往是并发问题	$C + K + \Psi \rightsquigarrow P_1 + P_9。$
59	使用 Actor 模型, 在无共享状态条件下实现并发	$K + R + C \rightsquigarrow P_9 + P_4。$
60	用共享的信息空间协调 workflow	$K + C + I + \Psi \rightsquigarrow P_5 + P_9 + P_4。$
61	识别并管理面对风险和批评时的本能防御反应	$K + R + \Psi \rightsquigarrow P_1$, 将直觉作为待验证信号。
62	不要依赖碰巧成立的行为、顺序或环境来编程	$K + \Sigma + G \rightsquigarrow P_1 + P_5。$
63	估算算法随输入规模增长的资源消耗	$R + G \rightsquigarrow P_3。$
64	用实际测试检验对算法性能的估算	$K + \Sigma + G \rightsquigarrow P_1 + P_{13}。$
65	尽早重构, 持续重构	$K + R + C + I + G \rightsquigarrow P_2 + P_3 + P_4 + P_{12}。$
66	测试用于验证行为、保护回归, 并发现 Bug	$K + I + \Sigma \rightsquigarrow P_5 + P_{12}。$
67	先用测试验证代码的使用方式和接口	$K + R + G \rightsquigarrow P_1 + P_4。$
68	先构建贯穿完整流程的可运行方案, 再逐步扩展	$K + C + \Sigma + G \rightsquigarrow P_1。$
69	为可测试性而设计	$K + C + \Sigma + G \rightsquigarrow P_1 + P_4 + P_5。$

#	Tip (中文概括)	路径
70	测试你的软件，避免把缺陷暴露给用户	$K + C + \Sigma + G \rightsquigarrow P_1$ 。
71	用基于属性的测试验证对系统行为的假设	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
72	保持简单，减少可被攻击的入口	$R + C + I + E \rightsquigarrow P_3 + P_{11}$ 。
73	快速应用安全补丁	$C + I + E + G \rightsquigarrow P_{10} + P_{11}$ 。
74	选择清晰准确的名称，需要时及时重命名	$K + I + G \rightsquigarrow P_5 + P_{12}$ 。
75	承认需求一开始通常并不明确	K 的需求领域实例。
76	帮助用户把模糊需求说清楚	$K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}$ 。
77	通过反复收集反馈和调整实现来逐步明确需求	$K + C + \Sigma + G \rightsquigarrow P_1 + P_{13}$ 。
78	和用户一起工作，站在用户角度思考	$K + \Sigma + G \rightsquigarrow P_1 + P_{12}$ 。
79	把变化频繁的策略作为可配置的元数据管理	$K + C + I + G \rightsquigarrow P_2 + P_5 + P_9$ 。
80	建立并使用统一的项目术语表	$K + I + R \rightsquigarrow P_5 + P_{12}$ 。
81	先识别问题和约束边界，再寻找突破方式	$K + \Sigma + G \rightsquigarrow P_1 + P_{13}$ 。
82	通过结对编程、代码评审或团队协作来编写代码	$K + R + I + \Psi \rightsquigarrow P_{12}$ 。
83	把敏捷落实为持续反馈、快速交付和持续改进的做事方式	$K + C + \Sigma + I + G \rightsquigarrow P_1 + P_2 + P_{13}$ 。
84	保持团队规模小而稳定	$R + I + \Psi \rightsquigarrow P_3 + P_{12}$ 。
85	把重要的改进和学习安排进日程，才能落实	$R + G + \Psi \rightsquigarrow$ 显式资源配置。
86	组建能够独立完成端到端工作的团队	$K + R + C + G + \Psi \rightsquigarrow P_4 + P_{12}$ 。
87	根据实际效果做事，别追逐时髦	$K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}$ 。
88	在用户真正需要时交付价值	$R + C + G \rightsquigarrow P_{13}$ ，价值时点由 G 给出。
89	让版本控制触发构建、测试和发布	$C + I + R + G \rightsquigarrow P_6 + P_8 + P_{10}$ 。
90	尽早测试、频繁测试，并自动执行测试	$K + C + R + \Sigma + G \rightsquigarrow P_1 + P_5 + P_8$ 。
91	所有测试跑完，编码才算完成	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ ，完成定义来自 G。
92	用故意制造错误的测试程序检验测试	$K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。
93	以状态覆盖率检验测试，别只看代码覆盖率	$K + I + \Sigma \rightsquigarrow P_5 + P_9$ 。

#	Tip（中文概括）	路径
94	每个已发现的 Bug 都要用自动化测试固定下来，避免再次出现	$K + I + R + G \rightsquigarrow P_6 + P_{12} + P_5。$
95	把手工流程自动化	$R + C + G \rightsquigarrow P_8。$
96	让用户获得满意和惊喜，交付可用价值	$K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}。$
97	为自己的成果署名并承担责任	$G + E + \Psi \rightsquigarrow$ 责任与可追溯性；KRCI 可解释其工程收益。
98	优先确保软件不造成伤害	$E \rightsquigarrow P_{11}； C + I$ 提高高损害行动的谨慎级别。
99	不要为恶意使用者提供便利	$E + C + I \rightsquigarrow P_{11}。$
100	分享成果，庆祝进展，持续改进和建设	$K + I + R + G + \Psi \rightsquigarrow P_{12} + P_{13}；$ 庆祝属于组织文化前提。

该表展示 P1–P13 在构造语料中的复用情况；它用于检查压缩性，不承担独立验证功能。

B 符号表

符号	含义
K	Knowledge bounded：认识有限
R	Resources bounded：资源有限
C	Control bounded：控制有限
I	Irreversibility：工程不可逆性
G	Goal：工程目标
E	Ethics：伦理边界
Σ	可学习结构条件
Ψ	心理学、组织行为与具体工具经验等辅助实证前提

参考文献

[1] Herbert A. Simon. “Rational Decision-Making in Business Organizations.” Nobel Memorial Lecture, 1978. <https://www.nobelprize.org/prizes/economic-sciences/1978/simon/lecture/>

[2] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956. <https://ashby.info/Ashby-Introduction-to-Cybernetics.pdf>

- [3] Rolf Landauer. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development*, 5(3):183–191, 1961. DOI: <https://doi.org/10.1147/rd.53.0183>
- [4] David L. Parnas. “On the Criteria To Be Used in Decomposing Systems into Modules.” *Communications of the ACM*, 15(12):1053–1058, 1972. DOI: <https://doi.org/10.1145/361598.361623>
- [5] Harold Abelson, Gerald Jay Sussman, with Julie Sussman. *Structure and Interpretation of Computer Programs*, 2nd ed. MIT Press, 1996. <https://mitpress.mit.edu/9780262510875/structure-and-interpretation-of-computer-programs/>
- [6] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM*, 32(2):374–382, 1985. <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
- [7] David H. Wolpert and William G. Macready. “No Free Lunch Theorems for Optimization.” *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. DOI: <https://doi.org/10.1109/4235.585893>
- [8] David Thomas and Andrew Hunt. *The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition*. Addison-Wesley, 2019. ISBN 9780135957059. <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>
- [9] The Pragmatic Bookshelf. “Pragmatic Programmer Tips: 100 Tips from the 20th Anniversary Edition.” <https://pragprog.com/tips/>
- [10] Adam Wiggins. *The Twelve-Factor App*. Last updated 2017. <https://12factor.net/>
- [11] Martin Thomson and David Schinazi. “Maintaining Robust Protocols.” RFC 9413, Internet Architecture Board, June 2023. DOI: <https://doi.org/10.17487/RFC9413>